Part 1

What is React Native?

React Native is a framework for building native mobile apps using Javascript and React. It allows developer to write mobile app code for iOS and Android using same codebase significantly reducing development time and cost. Unlike hybrid apps that run in a WebViews, React Native compiles to actual native components, providing performance and user experience similar to apps built with Swift (iOS) or Java/Kotlin (Android). React Native is used by companies Like facebook, Airbnb, Intagram and Tesla to build production-level mobile applications.

Difference b/w React & React Native Q1

While both use React's component-based architecture and Javascript, React Native differs from React in several key ways:

React	React Native Q2
→ Renders to HTML DOM	→ Renders to native mobile components
→ Uses HTML Elements	→ Uses native components
→ Uses CSS for Styling	→ Uses JS Object for Styling

→ Runs in web browsers	→ Run on iOS/Android devices
→ Uses ClassName for styling	→ Use Style prop with objects
→ Handle clicks with onClick	→ Handles touches with onPress
→ Uses React Router for navigation within web	→ Employs React Navigation for moble app navigation
→ Deployed on web serves, accessed through browsers	→ Deployed and submitted on app stores, accessed by instAlling in mobile.

Key Differerxe:

React Native does'nt browser or DOM - it commonicate with native plateforms APIs through a Javascript Bridge, make app truely native.

Plateforms React Native Target

- iOS (iPhone, iPad)
- Android (phone, tablets)
- web (with react-~~mative~~ native web)
- Windows (with react native -windows
- macOS (with react native- macOS)
- tvOS and Android TV (for Smart TVs)

Render UI Components

React

React Component → Virtual DOM → Browse DOM → HTML Render on Screen

React Native

React components → Virtual DOM → Native Bridge → Native iOS/ Android Comp → Render On device

1. You write React Component using Javascript
2. React Native creates a virtual DOM (Same as React web)
3. Javascript Bridge communicates b/w JS and native code
4. Native modules recieves instructions
5. Native UI components are rendered (actual iOS UIView or Android view)

React Native	Replaces	Native Equivalent
<View>	<div>	UIView(iOS), android.view.View
<Text>	<p>, <span>, <h1>	UITextView(iOS), TextView(Android)
<Image>	<img>	UIImage(iOS), ImageView(Android)
<ScrollView>	<div>; overflow:scroll	UIScrollView, ScrollView
<TextInput>	<input>, <textarea>	UIText field, Edit Text
<Button>	<button>	UIButton, Button
<TouchableOpacity>	<buttion> opacity effect	Custom touchable component
<FlatList>	with virtulization	UI TableView, Reycyler View
<Switch>	<input type=checkbox>	UISwitch, Switch

React native App Code Files Structure

```
MyFirstReactNativeApp/
├─ .expo
│  ├─ devices.json
│  ├─ README.md
├─ assests
│  ├─ icon.png
│  ├─ splash.png
│  ├─ adaptive-icon.png
├─ node_modules
├─ .gitignore
├─ App.js
├─ app.json
├─ index.js
├─ package.json
```

App.js

The main entry point of React Native app

→ First component that gets rendered when app starts

→ Contains app's root component

→ Similar to App.js in React webapp

→ where from building UI started

node_modules/

Contains all project dependencies and packages

- → Read native core libraries
- → Expo SDK package
- → Third-party libraries that are installed
- → All npm packages

--- Important -----

- → Never edit files in here
- → Never commit to Git
- → Can be regenerated with npm install
- → Usually very large

package.json

Package configuration and dependency management files

- → Lists of all project dependencies
- → Define npm scripts (start, build, etc.)
- → Store project metadata
- → Manages version numbers.

assets/

Stores static files like image, fonts, icons.

- → Any image, fonts or media files needed in Project.

app.json

Expo and app configuration file.

→ App name and description
→ App icon and splash screen
→ iOS and Android Specific Settings
→ Permission Required
→ SDK versions

Recommended Structure

```
MyApp
├ App.js                # Root component
│
├── src/                # All source code
│   ├── components/     # Reusable Comp
│   │   ├── common/     # share UI comp
│   │   ├ Button.js
│   │   ├ Card.js
│   │   └ Input.js
│   └── features/       # feature specific comp
│       └ UserCard.js
│       └ PostCard.js
│   │
│
```

```
|   |
|   ├── screens/            # screens/ pages comp
|   |   ├── HomeScreen.js
|   |   ├── ProfileScreen.js
|   |   ├── Settings.js
|   |
|   ├── navigation/         # navigation setup
|   |   ├── AppNavigator.js
|   |   ├── AuthNavigator.js
|   |   ├── TabNavigator.js
|   |
|   ├── services/           # API calls & External services
|   |   ├── api.js
|   |   ├── authService.js
|   |   ├── userService.js
|   |
|   ├── utils/              # Helper functions
|   |   ├── validation.js
|   |   ├── constants.js
|   |
|   ├── hooks/              # Custom React Hook
|   |   ├── useAuth.js
|   |   ├── useFetch.js
|   |
|   └── contexts/           # React context provider
|       ├── AuthContext.js
|       ├── ThemeContext.js
```

```
│  ├─ styles/            # Global Styles
│  │  ├─ colors.js
│  │  ├─ typography.js
├─ assets/               # static files
│  ├─ images/
│  ├─ icons/
│  ├─ fonts/
│
├─ app.json              # Expo config
├─ packge.json           # dependencies
└─ node_module/          # installed packages
```

Styling

Rect web CSS	**React Native Style-Sheet**
.container{ display:flex justify-content:center background-color:#000 }	const style=StyleSheet.create({ contain:{ flex: 1 justifyContent:'center', } });

Rect web CSS

```
.container{
display:flex
justify-content:center
background-color:#000
}
```

React Native Style-Sheet

```
const style=StyleSheet.
  create({
  contain:{
    flex: 1
    justifyContent:'center',
  }
});
```

→ CSS file (.css	→ Java script objects
→ kebab-case {background-color}	→ camelCase (backgroundColor)
→ string values ("20px")	→ Number value 20
→ padding: 20px	→ padding: 20
→ Many units	→ Mainly unitless number
→ className prop	→ style prop
→ External stylesheet	→ Inline StyleSheet.create
→ CSS pseudo-classes (:hover)	→ component states

→ No CSS file in React Native
→ No class names like web
→ styles are JS objects
→ ~~Styles~~ Most layout uses Flexbox
→ No unit needed

Role of Flexbox in React Native

Flexbox is PRIMARY layout system in React Native

unlike (float, grid, position).
React native uses Flexbox for almost all layout.

Key flex box properties in React Native

```
const style = StyleSheet.create({
    container: {
        flex: 1,
        flexDirection: 'column'
        justifyContent: 'center'
    }
});
```

→ flexDirection defaults to 'column' (vertical)

→ In web, it defaults to 'row' (horizontal)

Property	**Values**	**Purpose**
flexDirection	'row', 'column'	Layout Direction
justifyContent	'center', 'flex-start', 'flex-end', 'space-between'	Main axis alignment
alignItems	'center', 'flex-start', 'flex-end', 'stretch'	cross axis alignment
flex	1, 2, 3, ...	Take propotional space

Why Flexbox is Essential:

→ No position: absolute layout system like web
→ No float or clear
→ flexbox handles responsive layout automatically
→ works consistently across devices